

Strings: analisar e transformar texto

Questão 1 · Uma sequência imutável

Regista os resultados e explica como criaste a nova palavra. Em que difere substituir a variável de modificar um carácter da string?

Original: P n; yth; nohtyP; Original: Python; Novo: Jython. Com Gato e R: G o; ato; otaG; Original: Gato; Novo: Rato. `len(texto)` é 4.

A expressão `'R' + texto[1:]` concatena uma letra com a fatia restante. Criar uma nova string e atribuí-la a uma variável não modifica a string original.

Questão 2 · Normalizar para comparar

Compara os resultados de `in` e `find`. Explica por que normalizar o nome não basta se o termo de pesquisa continuar com maiúsculas.

Original normalizado: `'ana silva'`. A presença de `ana` é `True`. `find('Silva')` devolve `-1`. Mudando para `find('silva')`, devolve `4`. Com `'ANA SOUSA'`, o nome é `'ana sousa'`: `ana` continua presente, mas `silva` não é encontrado (`-1`).

A pesquisa distingue maiúsculas de minúsculas. Tanto o texto como o termo devem seguir a mesma normalização para uma comparação sem distinção de caixa.

Strings: analisar e transformar texto

Questão 3 · Contar palavras e caracteres

Que resultados mudam com os espaços? Explica por que útil. mantém o ponto dentro do token.

Lista original: ['python', 'é', 'útil.', 'python', 'é', 'claro.']. Existem seis tokens e duas ocorrências de python. Espaços adicionais não mudam estes resultados, porque split() sem argumento agrupa espaços em branco.

Uma string só com espaços produz uma lista vazia, zero palavras e zero ocorrências. A pontuação continua nos tokens: split separa por espaços, não remove pontos.

Questão 4 · Interpretar um registo simples

Regista os três resultados. Explica a diferença entre validar a estrutura do registo e validar a nota.

'102; Rui ;12' produz (102, 'Rui', 12.0). '103;Eva' mostra Número de campos inválido. '104;Leo;25' deve ser rejeitada por nota fora do intervalo.

```
linha = '104;Leo;25'
campos = linha.split(';')
if len(campos) == 3:
    nota = float(campos[2])
    if 0 <= nota <= 20:
        aluno = (int(campos[0]), campos[1].strip(), nota)
        print(aluno)
    else:
        print('Nota inválida')
else:
    print('Número de campos inválido')
```

A estrutura verifica a existência de três campos; o intervalo verifica uma regra sobre o valor convertido. Assume-se conteúdo numérico válido quando há três campos, como indica a atividade.

Strings: analisar e transformar texto

Questão 5 · Desafio final - uma solução possível

```
def pesquisar_nomes(nomes, termo):
    termo = termo.strip().lower()
    if not termo:
        return []
    encontrados = []
    for nome in nomes:
        if termo in nome.lower():
            encontrados.append(nome)
    return encontrados

nomes = ['Ana Silva', 'Bruno', 'Mariana', 'Rui']
termo = ' ANA '
print(pesquisar_nomes(nomes, termo))
print(nomes)
```

'ANA ': ['Ana Silva', 'Mariana']; 'rui': ['Rui']; 'xyz' e apenas espaços: []. Mariana contém a sequência ana no fim. A pesquisa usa nomes normalizados apenas na comparação; conserva a grafia original na nova lista. nomes não sofre alterações.